

Proyecto Final de Procesamiento Digital de Señales

Transformador de imágenes para hacerlas más livianas antes de enviarlas a IA

Alejandro Apodaca Córdova — Matrícula m041852

Gael Calderón Robles — Matrícula m042449

Ingeniería Mecatrónica, CETYS Universidad, Campus Mexicali

2 de junio de 2026

Resumen

El proyecto consiste en una página web que toma una imagen original y genera una versión JPG más liviana para enviarla a un chat o sistema de inteligencia artificial. La idea principal se resume así: *la misma imagen visual, menos peso*. El sistema procesa la imagen en el navegador, reduce su dimensión máxima, la comprime como JPEG y muestra tres datos fáciles de entender: tamaño original, tamaño optimizado y porcentaje de ahorro. Aunque la interfaz final es simple, el funcionamiento interno aplica conceptos de procesamiento digital de señales: adquisición de una señal visual, remuestreo espacial, codificación con pérdida, medición de error y modelización del ahorro.

1 Problema de investigación

Muchas aplicaciones modernas permiten subir imágenes a modelos de IA. El problema es que una imagen tomada con un teléfono puede pesar varios megabytes, aunque para explicar una idea en un chat no siempre se necesita toda esa resolución. Enviar imágenes pesadas aumenta el tiempo de carga, el ancho de banda y el tamaño de los datos si la imagen se convierte a base64 para transmitirse por una API.

Por eso se plantea el siguiente problema de investigación: **¿cómo reducir el peso de una imagen antes de enviarla a un sistema de IA, manteniendo una versión visualmente útil para el usuario?**

Las variables principales del problema son:

- Tamaño original de la imagen, medido en bytes.
- Tamaño de la imagen optimizada, medido en bytes.
- Porcentaje de ahorro.
- Dimensión máxima permitida después del remuestreo.
- Calidad JPEG usada para la compresión.
- Error entre la imagen procesada y la imagen codificada.

La relevancia del problema está en que optimizar imágenes antes de subirlas reduce recursos de red y mejora la experiencia de uso. En ingeniería, esto permite tomar una decisión técnica con datos y no solo con percepción visual.

2 Preguntas de investigación

Para guiar el proyecto se definieron tres preguntas:

1. ¿Cuánto peso se puede ahorrar al remuestrear una imagen y codificarla como JPEG?
2. ¿Cómo mostrar ese ahorro de forma simple?
3. ¿Qué limitaciones aparecen cuando se usa compresión JPEG para preparar imágenes destinadas a sistemas de IA?

3 Marco teórico

Una imagen digital puede entenderse como una señal discreta bidimensional $x[m, n]$. Cada coordenada representa una muestra espacial y cada muestra contiene información de color. Cuando se reduce la resolución de la imagen, se reduce el número total de muestras. Este proceso se conoce como remuestreo espacial.

El prototipo también puede convertir la imagen a luminancia usando:

$$Y = 0.299R + 0.587G + 0.114B$$

Esta operación resume la información de color en brillo, lo cual puede ser útil cuando el color no es esencial para el análisis.

Después del remuestreo, la imagen se codifica como JPEG. JPEG usa compresión con pérdida: reduce información para obtener un archivo más pequeño. Para medir el error se calcula el error cuadrático medio:

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2$$

y la relación señal a ruido pico:

$$PSNR = 10 \log_{10} \left(\frac{255^2}{MSE} \right)$$

En la interfaz final no se muestra PSNR para no confundir al usuario, pero sí se calcula internamente y se usa como evidencia técnica en el reporte.

4 Herramienta moderna desarrollada

Se creó una aplicación web con Vite, React, TypeScript y Tailwind CSS. La herramienta funciona en el navegador, por lo que la imagen se transforma antes de llegar a un servidor. Esto es importante porque el objetivo del proyecto no es solo comprimir una imagen, sino evitar que una imagen innecesariamente pesada sea enviada a una aplicación de IA.

La interfaz final se diseñó para ser muy simple:

- El usuario sube una imagen.
- La página genera una versión optimizada.
- Se muestran tres métricas: original, optimizada y ahorro.
- El usuario puede descargar el JPG final.



Figura 1: Interfaz web del transformador de imágenes.

5 Metodología

La metodología aplicada fue la siguiente:

1. Adquirir una imagen real desde el navegador mediante carga de archivo.
2. Decodificar la imagen y registrar sus dimensiones y bytes originales.
3. Remuestrear la imagen para limitar su lado mayor a 1280 pixeles.
4. Codificar la imagen resultante como JPEG con calidad 72%.
5. Comparar el tamaño original contra el tamaño optimizado.
6. Calcular porcentaje de ahorro, expansión base64 evitada, MSE y PSNR.
7. Mostrar al usuario solo las métricas más importantes para la decisión: original, optimizada y ahorro.

Esta metodología cumple con la rúbrica porque define variables, aplica un método de investigación, ejecuta un experimento reproducible y permite interpretar los datos obtenidos.

6 Diseño experimental

Para validar el prototipo se usó una imagen de prueba reproducible incluida en el repositorio como `fixture-1.png`. Las variables controladas fueron:

- Calidad JPEG: 72%.
- Dimensión máxima: 1280 px.
- Formato de salida: JPG.
- Procesamiento local en navegador.

Las variables de respuesta fueron:

- Tamaño original.
- Tamaño optimizado.
- Porcentaje de ahorro.
- Relación de compresión.
- Base64 evitado.
- PSNR.

7 Resultados

Con la imagen de prueba, el sistema obtuvo los resultados de la Tabla 1. El caso demuestra que una imagen puede mantenerse útil visualmente y reducirse de forma considerable para transporte digital.

Cuadro 1: Resultados de la prueba reproducible con `fixture-1.png`.

Métrica	Resultado
Resolución original	1600 × 1000 px
Resolución optimizada	1280 × 800 px
Tamaño original	572 KB
Tamaño optimizado JPG	91 KB
Ahorro directo	84.1%
Relación de compresión	6.31x
Base64 evitado	642 KB
PSNR	36.2 dB

El patrón principal es claro: la reducción de resolución y la compresión JPEG disminuyen drásticamente el peso del archivo. Esto responde directamente al concepto de la página: **la misma imagen puede ser mucho más liviana**. Para un usuario no técnico, el porcentaje de ahorro es la métrica más importante. Para justificarlo técnicamente, el reporte conserva las métricas de DSP y modelo de datos.

8 Predicción y modelización

La rúbrica 24ICE05 pide incluir predicción y modelización en problemas de ingeniería. En este proyecto se modeló la expansión base64 con la regla:

$$B_{64} = 4 \left\lceil \frac{B}{3} \right\rceil$$

donde B es el tamaño en bytes del archivo binario. Este modelo predice cuánto crece una imagen al serializarse para transporte textual en una API.

También se modeló el porcentaje de ahorro:

$$Ahorro = \frac{B_{original} - B_{optimizado}}{B_{original}} \times 100$$

Con este modelo la aplicación puede informar al usuario cuántos recursos se reducen antes de subir la imagen.

9 Limitaciones técnicas

El proyecto también reconoce límites importantes:

- JPEG no conserva transparencia; una imagen PNG con canal alfa puede cambiar visualmente.
- La compresión puede afectar texto pequeño o detalles finos.
- PSNR mide diferencia numérica, pero no garantiza que un modelo de IA interprete exactamente igual la imagen.
- La compatibilidad de formatos depende del navegador.
- El modelo de unidades de contexto es una aproximación, no el conteo interno de un proveedor específico de IA.

Para reducir estas limitaciones se podrían agregar validaciones futuras como OCR para texto pequeño, SSIM para comparación perceptual y pruebas con modelos multimodales reales.

10 Alineación con las rúbricas

10.1 24ICE04: investigación de problemas de ingeniería complejos

El reporte cumple con 24ICE04 porque identifica un problema real, justifica su relevancia, plantea preguntas de investigación, selecciona métodos adecuados, aplica un experimento, analiza datos y genera conclusiones válidas. El problema no se trata solo como una página web, sino como una decisión de ingeniería: cuánto peso se reduce y qué costo técnico se evita.

10.2 24ICE05: herramientas modernas de ingeniería y TI

El reporte cumple con 24ICE05 porque se creó una herramienta moderna de TI usando React, TypeScript y procesamiento en navegador. La herramienta se seleccionó porque permite transformar la imagen antes de enviarla al servidor. También se reconocen limitaciones técnicas y se usan modelos para predecir ahorro de bytes y expansión base64.

Cuadro 2: Resumen de cumplimiento.

Criterio	Evidencia
Definición del problema	Imágenes pesadas aumentan costo de envío a chats o sistemas de IA.
Método de investigación	Remuestreo, JPEG, medición de bytes, MSE y PSNR.
Diseño experimental	Fixture reproducible, calidad fija, dimensión máxima y métricas de respuesta.
Análisis de datos	Comparación entre tamaño original, tamaño optimizado y ahorro.
Herramienta moderna	Aplicación web con Vite, React, TypeScript y Canvas.
Modelización	Fórmulas para ahorro porcentual y expansión base64.
Limitaciones	Transparencia, texto pequeño, PSNR, formatos del navegador y modelo aproximado.

11 Conclusiones

El proyecto demuestra que aplicar procesamiento digital de señales a imágenes puede resolver un problema actual: reducir el peso de archivos antes de enviarlos a sistemas de IA. Con una configuración simple de 1280 px y calidad JPEG de 72%, la prueba obtuvo un ahorro de 84.1%. La interfaz final se mantuvo deliberadamente simple para que cualquier persona entienda el beneficio sin conocer MSE, PSNR o base64.

La conclusión principal es que la optimización local permite ahorrar recursos sin complicar la experiencia del usuario. El valor del proyecto está en unir una técnica de DSP con una necesidad moderna de aplicaciones web e inteligencia artificial.

12 Reflexión sobre la competencia evaluada

La competencia 24ICE04 se relaciona con investigar problemas complejos de ingeniería y obtener conclusiones válidas a partir de datos. En este proyecto se practicó esa competencia al definir un problema, controlar variables, medir resultados e interpretar el ahorro obtenido.

La competencia 24ICE05 se relaciona con crear, seleccionar, aplicar y reconocer limitaciones de herramientas modernas de ingeniería y TI. En este proyecto se aplicó al construir una herramienta web funcional, seleccionar tecnologías adecuadas y reconocer

que la compresión JPEG no siempre es suficiente para todos los casos. La reflexión final es que una herramienta moderna no debe ser complicada para ser útil: debe resolver un problema real, mostrar datos claros y reconocer sus límites.

Publicación y repositorio

La página publicada del proyecto se encuentra en:

`https://capiz.apoapps.com`

El código fuente se encuentra en GitHub:

`https://github.com/apoapps/proyecto-final-procesamiento-digital-senales`

Referencias

- [1] R. C. Gonzalez y R. E. Woods, *Digital Image Processing*. Pearson.
- [2] A. V. Oppenheim y R. W. Schaffer, *Discrete-Time Signal Processing*. Pearson.
- [3] G. K. Wallace, "The JPEG Still Picture Compression Standard", *Communications of the ACM*.
- [4] MDN Web Docs, *Canvas API*.